Chapter 6

Role-Based Authorization in Spring Security

From an authenticated user to a permissioned one — wiring roles, authorities, and access rules with Spring Security.

1. What is Authorization?

Authentication checks **who the user is**, while authorization checks **what the user is allowed to access**. For example, ADMIN can access admin endpoints, but a DONOR should not be able to. If a DONOR tries to access an ADMIN-only endpoint, Spring Security returns **403 Forbidden**.

AUTHENTICATION	AUTHORIZATION
“Who are you?”	“What are you allowed to do?”
JWT validates the user	Authorities / roles decide access

2. Implementing UserDetails

Our `User` class is a normal entity containing fields such as email, password, and role. To provide this security-related information to Spring Security in its expected format, we implement the `UserDetails` interface:

```
public class User implements UserDetails
```

`UserDetails` provides methods through which Spring Security can access the username, password, authorities, and account status of the user. In our project, we use the user's **email as the username**:

```
@Override
public String getUsername() {
    return email;
}
```

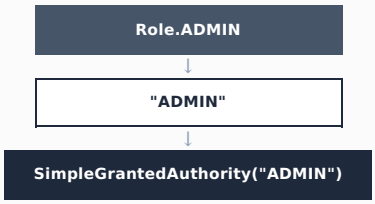
3. GrantedAuthority

Spring Security represents roles and permissions using the `GrantedAuthority` interface. Our project already has a custom `Role` enum:

```
ADMIN
DONOR
NGO
VOLUNTEER
```

Spring Security expects roles in the form of `GrantedAuthority`. `SimpleGrantedAuthority` is a ready-made class provided by Spring Security that implements this interface, so we convert our role:

```
SimpleGrantedAuthority authority =
        new SimpleGrantedAuthority(role.name());
```



4. getAuthorities()

`UserDetails` contains the following method:

```
@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    SimpleGrantedAuthority authority =
            new SimpleGrantedAuthority(role.name());

    return List.of(authority);
}
```

`Collection<? extends GrantedAuthority>` means the method can return a collection such as a `List` or `Set` containing objects that implement `GrantedAuthority`. In our project, each user currently has one role, so we return `List.of(authority)`.

5. Adding Authorities to Authentication

When the JWT is validated inside `JwtFilter`, we create an Authentication object:

```
UsernamePasswordAuthenticationToken authentication =
        new UsernamePasswordAuthenticationToken(
                user,
                null,
                user.getAuthorities()
        );
```

The three values represent:

ARGUMENT	MEANING
`user`	Current authenticated user
`null`	Credentials
`user.getAuthorities()`	User's roles / authorities

We then store this Authentication object inside the `SecurityContext`:

```
SecurityContextHolder
        .getContext()
        .setAuthentication(authentication);
```

Now Spring Security knows both who the current user is, and what authorities that user has.

6. SecurityConfig

`SecurityConfig` contains the security rules of our application. `@Configuration` tells Spring that this class contains application configuration; inside it, we create the `SecurityFilterChain`:

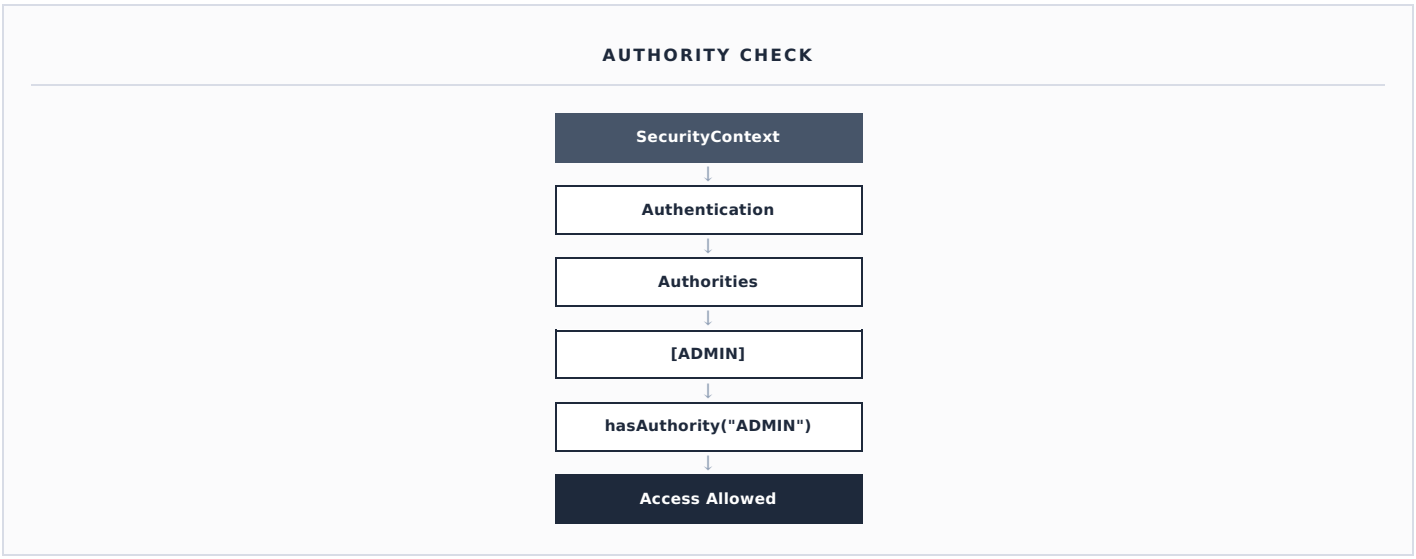
```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

    http
        .csrf(csrf -> csrf.disable())

        .authorizeHttpRequests(auth -> auth

            .requestMatchers("/users/admin/**")
            .hasAuthority("ADMIN")

            .requestMatchers("/users/**")
            .permitAll()

            .anyRequest()
            .authenticated()
        )

        .addFilterBefore(
            jwtFilter,
            UsernamePasswordAuthenticationFilter.class
        );

    return http.build();
}
```

7. hasAuthority()

This rule:

```
.requestMatchers("/users/admin/**")
.hasAuthority("ADMIN")
```

means any request to `/users/admin/**` can only be accessed by a user whose Authentication object contains the `ADMIN` authority. Spring checks the Authentication object stored inside the `SecurityContext`:



If the authority is instead `[DONOR]`, authorization fails and Spring returns **403 Forbidden**.

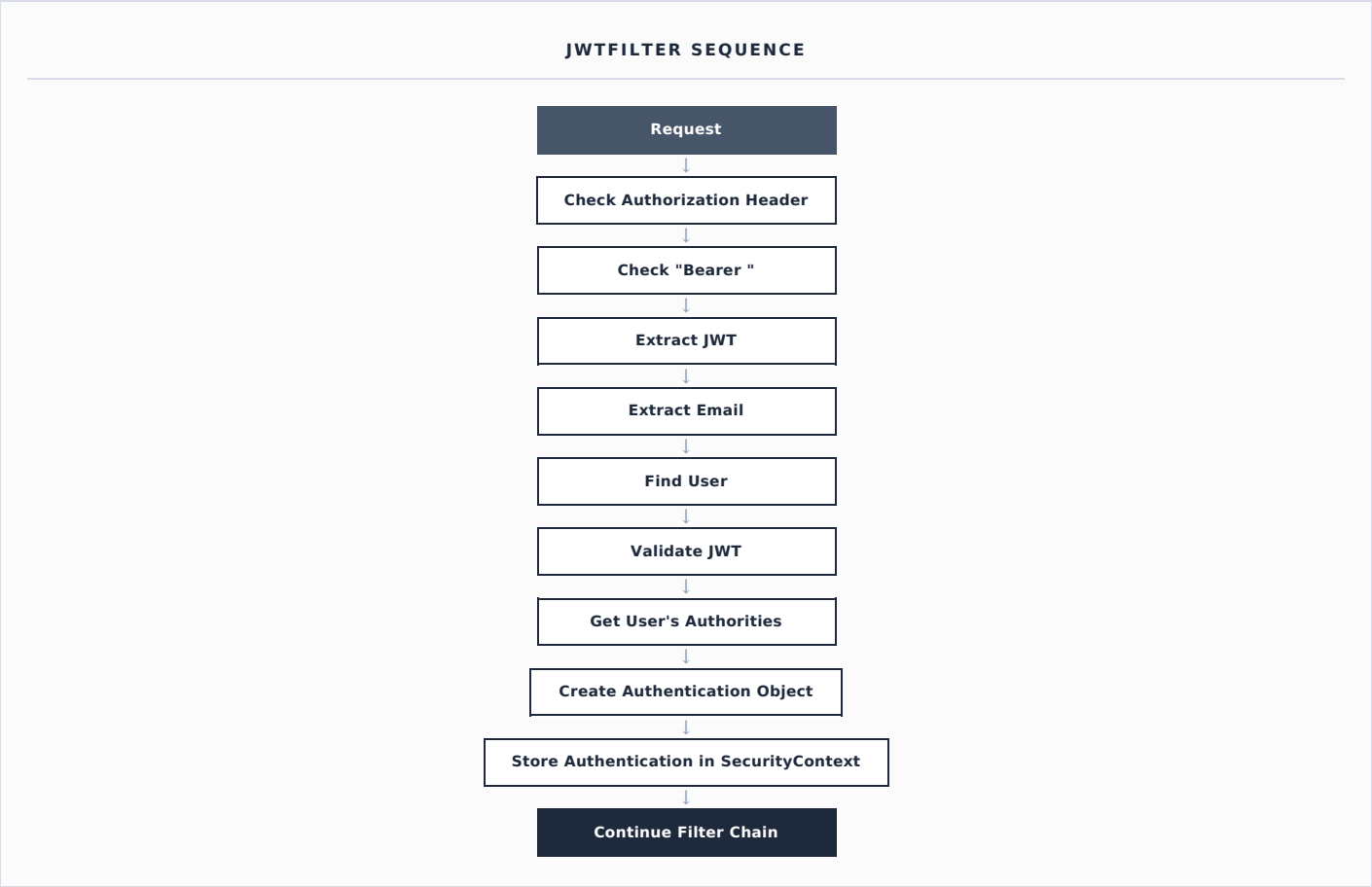
8. permitAll()

```
.requestMatchers("/users/**")
.permitAll()
```

This means matching endpoints can be accessed without authentication — useful for endpoints such as registration and login.

9. JwtFilter

Our custom `JwtFilter` extends `OncePerRequestFilter`, which allows the filter to execute once for each HTTP request. We override `doFilterInternal()`, which receives `HttpServletRequest`, `HttpServletResponse`, and `FilterChain`. The JWT filter performs the following flow:

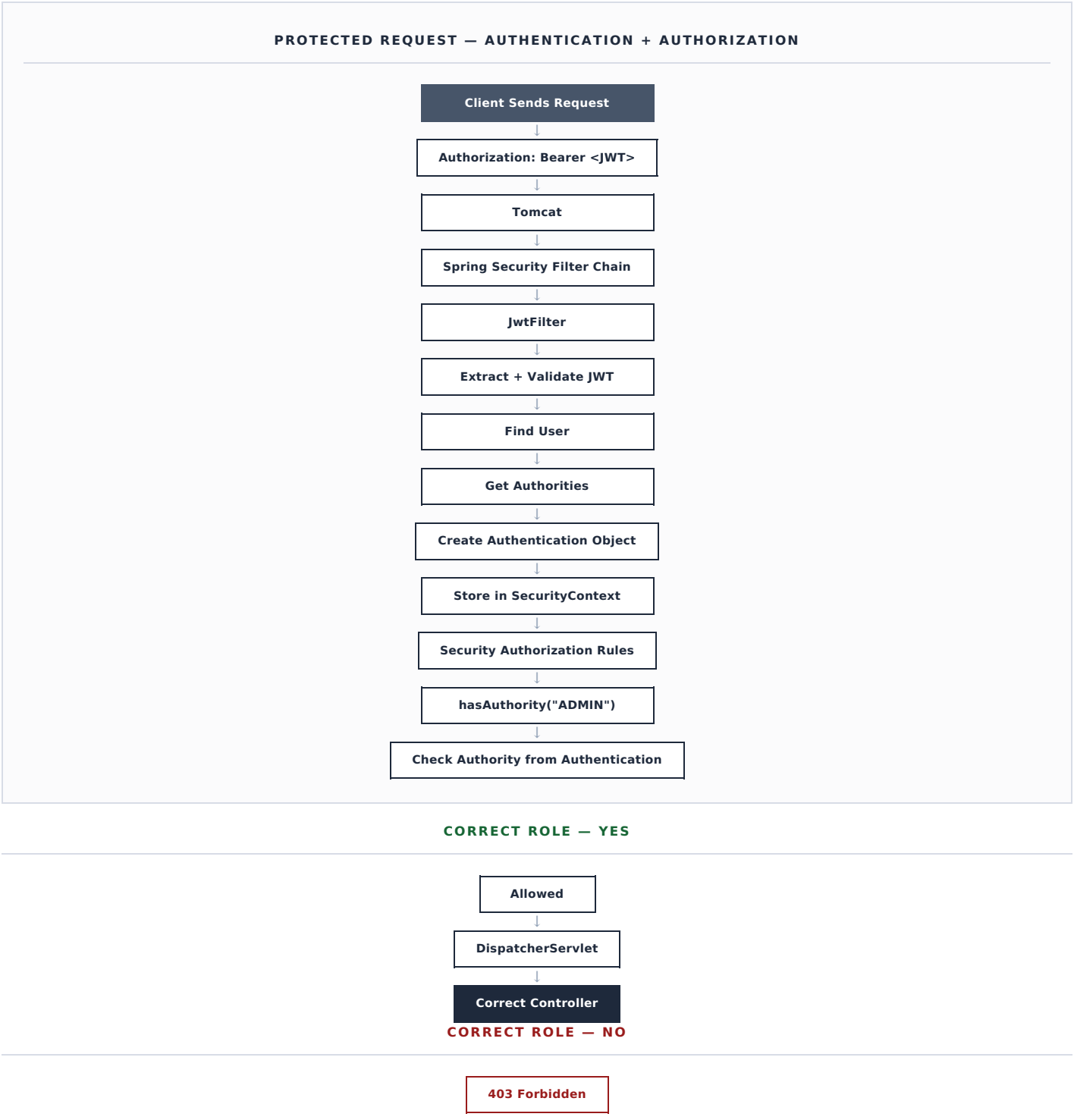


10. Adding JwtFilter to Security

```
.addFilterBefore(
    jwtFilter,
    UsernamePasswordAuthenticationFilter.class
);
```

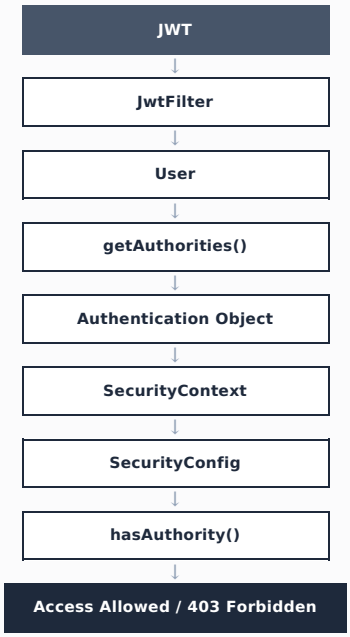
This tells Spring Security to execute our `JwtFilter` before `UsernamePasswordAuthenticationFilter`. Therefore, our JWT can be validated and the Authentication object can be placed in the SecurityContext before authorization rules are checked.

11. Complete Authorization Flow



Final Concept

The most important flow to remember is:



AUTHENTICATION VS AUTHORIZATION

AUTHENTICATION — "WHO ARE YOU?"	AUTHORIZATION — "WHAT ARE YOU ALLOWED TO DO?"
JWT validates the user	Authorities / roles decide access

CHAPTER SUMMARY

At this point, the basic role-based authorization mechanism is complete. Our `User` implements `UserDetails` and exposes its role as a `GrantedAuthority`; `JwtFilter` attaches those authorities to the `Authentication` object stored in the `SecurityContext`; and `SecurityConfig` uses `hasAuthority()` and `permitAll()` to decide, for every request, whether access is allowed or rejected with a 403.

> *The next step is to apply authorization rules to the actual FoodBridge features for DONOR, NGO, VOLUNTEER, and ADMIN.*